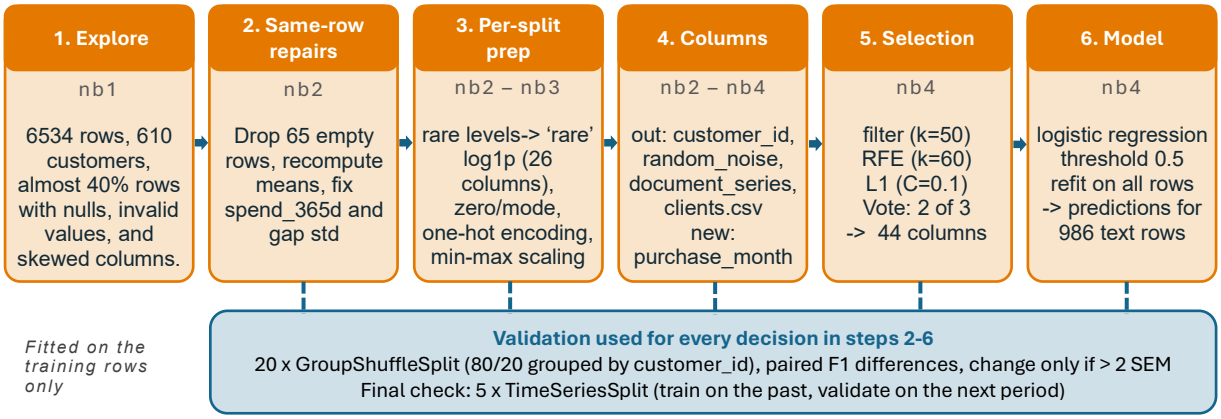
# Homework Group 09 – Predicting 90-day repeat purchases

MACHINE LEARNING 2026 | PIPELINE STRUCTURE

## Goal

Our task is to predict a variable called **repeat_purchase_90d**, which tells if a customer will buy again within 90 days of the current purchase (1) or not (0). Each row corresponds to one purchase and the same customer may be associated with more than one (453 of the 610 customers are linked to several purchase rows). The purchases in the test set (test.csv) occur later than every purchase present in the training set (train.csv); in fact, the first runs from 05/2025 up to 10/2026 and the latter records dates from 07/2015 up to 02/2025. Moreover, 94.5% of its rows are related to customers already analyzed before in the set train.csv.

Two rules dictate this pipeline: **(1)** anything that is learned from data that was given to us (fill values, rare levels, encoder, scaler, selected columns, model) is first fitted on the training rows of each split only and then applied to its validation rows or to test.csv file; **(2)** every choice between alternatives is **measured** and not just assumed: each candidate is scored on the same 20 customer-grouped splits, only replacing the simpler option when the paired F1 difference is larger than two standard errors.




## 1. Exploration (nb1)

Right after taking a quick look, we noticed that the target is balanced (55.9% repeats) and after some data exploration, we found unconventional information: 65 rows are empty in all feature columns, an oddly specific -4 value appears in two day columns, negative values in prior_mean_purchase_value and spend_365d when these variables shouldn't accept numbers below 0, strongly right-skewed variable distributions and count columns with too many outliers.

Additionally, we observed that three invoice totals are almost perfectly correlated (0.97-1) and the purchase windows of 30/90/180 days (which are nested) also have high correlation (0.75-0.95). Moreover, customers who buy again tend to have more previous purchases and shorter gaps between them. 40 levels of the 80 that transport_method has, are occupied by a single customer.

## 2. Cleaning and pre-processing (nb2, nb3)

**Same-row repairs** (they learn nothing, so they are applied identically to train and test): drop the 65 empty rows; recompute prior_mean_purchase_value (272 rows) and prior_mean_gap_days (205 rows); rebuild spend_365d when prior_total_spend or prior_purchase_count cover the whole history; set the standard deviations to 0 when there is only one observation; turn the remaining impossible values into missing (clipping, dropping or flagging scored the same).

**Repeated customers:** keeping one row per customer would leave 607 rows and change the repeat rate from 0.56 to between 0.07 and 0.29. We decide to keep every row and group every split by customer_id, so a customer is never on both sides of a split.

**Inside each split:** rare transport_method levels (fewer than 5 training customers) become 'rare'; log1p on the 26 skewed columns (0.0083 ± 0.0018 above keeping the values); missing values filled with zero in numeric columns and with the mode in categorical columns (nine imputation plans were compared, all within the noise); min-max scaling (best of four scalers; without scaling the solver didn't converge).

**Encoding (nb3):** 4 encoders x 4 scalers scored between 0.7005 and 0.7074 F1, all within one standard error. We use one-hot encoding , because target encoding turns single-customer levels into the customer's own repeat rate, i.e. it memorises customers. We chose one-hot with min-max scaler.

## 3. Feature selection (nb4)

For this part, we chose to use one method of each family from the course (week 4 notebook): a filter (Cramér's V or point-biserial relevance to the target, skipping columns correlated above 0.70 with one already kept), a wrapper (RFE with logistic regression) and an embedded method (L1-penalised logistic regression). Their sizes were chosen on the grouped splits (filter k=50, RFE k=60, L1 with the strongest penalty that did not lose F1, C=0.1). Because the families often disagree (e.g. prior_mean_gap_days is kept by RFE and L1 in every split but by the filter in 0), a column is kept when at least 2 of the 3 select it, inside every split. The final set is the 44 columns kept in most of the 20 splits. The vote matches all 94 columns on the grouped splits (-0.0013 ± 0.0012; 6/20 within the noise) with about half of the columns and beats them in all five time-ordered folds.

## 4. Model and assessment (nb4)

**Model**: logistic regression, a simple, interpretable model from the course that converges in at most ~100 iterations on the scaled data, with the default 0.5 threshold (0.4 gave almost the same F1, 0.7641 vs 0.7610).
**Metrics**: F1 for the 'repeat' class, because it balances precision (customers contacted who really come back), recall (returning customers we find) and ROC-AUC, which does not depend on the threshold. Both are compared with always predicting 'repeat', which already reaches an F1 score of 0.69-0.72.
**Assessment**: 20 grouped 80/20 splits for every decision, then 5 TimeSeriesSplit folds which mimic the test set (train on the past, predict the next period, mostly known customers).

| | Grouped splits (20) | | Time-ordered folds (5) | |
|---|---|---|---|---|
| **Model** | **F1** | **ROC-AUC** | **F1** | **ROC-AUC** |
| Always predict 'repeat' (baseline) | 0.6945 | 0.50 | 0.7206 | 0.5 |
| Logistic regression, all 94 columns | 0.7192 | 0.7683 | 0.7509 | - |
| **Final model: vote, 44 columns** | 0.7179 | 0.7682 | 0.7597 | 0.7957 |

The final model is the best of the three in 4 of 5 periods (F1 0.72-0.79) and its Kaggle public score was about 0.81, in line with the most recent period (0.796).

## 5. Limitations

Most of the times, the target can be rebuilt from the data: it equals 'the same customer has another row within 90 days' in 99% of training rows, and 780 of the 986 test rows have, inside test.csv, a later purchase made by the same customer. Using it would imply using future purchases and information from test.csv, which is something impossible at prediction time and above all forbidden by this project's guidelines, so we did not look forward to any feature (we checked that every window counts past purchases only). Our estimates are still optimistic: the feature set was chosen on all training rows and in the time-ordered folds the last training rows have targets that depend on purchases inside the validation period.

## 6. Summary of method implementation choices

| Step | Choice | Why it is appropriate |
|---|---|---|
| **Repeated customers** | Keep all rows, split by customer_id | One row per customer loses 90% of the data and distorts the target; grouping stops the model scoring on customers it has already seen |
| **Impossible values** | Recompute from same-row formulas | The totals and counts on the row give the exact value; learns nothing, so it is applied to test.csv safely |
| **Missing values** | Zero / Mode, fitted per split | Uses one global value; all nine plans were within noise |
| **Outliers** | Log1p on 26 skewed columns | Keeps every row and reduces the pull of extreme values; best of five treatments |
| **Categories** | Rare level grouped, one-hot encoding | Stops single-customer levels acting as identifiers; no order is imposed on the levels |
| **Scaling** | Min-max | Needed for the solver to converge and for L1/RFE to compare columns; best of four scalers |
| **Identifiers** | customer_id, random_noise, document_series, clientes.csv out | They identify customers or periods, or carry no signal, and would not generalise to new data |
| **Selection** | Vote of filter, RFE and L1 (2 of 3) | The three families disagree; the vote keeps about half of the columns. |
| **Model** | Logistic regression, threshold 0.5 | Simple, interpretable and stable; probabilities support ROC-AUC and threshold analysis |
| **Assessment** | 20 grouped splits + 5 time-ordered folds, F1 and AUC | Matches repeated customers and later test period; F1 suits the 'repeat' class, AUC is threshold-free |